



Actividad práctica en parejas

Objetivo: aplicar herencia para eliminar código duplicado, usando `extends`, `super(...)`, la sobrescritura de métodos y un criterio para elegir la visibilidad de cada miembro.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es 05-herencia/actividad.

- `Libro.java` y `Electronico.java` están completos y funcionando. Se modifican en la parte 2.
- `Producto.java` es un esqueleto con los TODO de la parte 2.
- `Alimento.java` no existe todavía. Se escribe en la parte 3.
- `Tienda.java` es el programa que muestra el catálogo. No se modifica, salvo para descomentar la parte 3 cuando el enunciado lo indique.

Parte 1: ver el problema

`Electronico.java` nació como copia de `Libro.java`. El que hizo la copia adaptó casi todo.

Ejecutar `Tienda` tal como viene y mirar el detalle del electrónico. Responder entre los dos, por escrito y en pocas palabras.

1. El catálogo dice que el autor de la notebook es Asus. ¿En qué archivo está el error y cómo llegó hasta ahí? ¿El compilador podía avisar algo?
2. Marcar en el código qué atributos y qué métodos de `Libro` y `Electronico` son idénticos entre sí, y cuáles son propios de cada clase.
3. La tienda quiere sumar alimentos al catálogo. Estimar cuántas de las líneas que ya existen habría que volver a escribir para tener un `Alimento.java` hecho con copia, como hasta ahora.

Parte 2: arreglar con herencia

Los pasos están marcados como TODO dentro de los archivos. En orden.

- TODO 1 a 3 (`Producto`): los atributos comunes, el constructor con las mismas validaciones que hoy tienen `Libro` y `Electronico`, y los getters. Los atributos quedan `private`.



PROGRAMACIÓN ORIENTADA A OBJETOS

- TODO 4 (Producto): `mostrarDetalle()` con el encabezado común del catálogo (código, nombre, precio y precio final con IVA).
- TODO 5 (Libro): heredar de `Producto` con `extends`, borrar lo que quedó duplicado, llamar a `super(...)` como primera instrucción del constructor y sobrescribir `mostrarDetalle()` reutilizando `super.mostrarDetalle()` para el encabezado y agregando el autor y el ISBN.
- TODO 6 (Electronico): lo mismo. Cada clase escribe solo sus líneas propias, así que el autor fantasma desaparece solo.

Parte 3: verificar y discutir

Descomentar la parte 3 de `Tienda` y escribir `Alimento.java` completo, sin esqueleto. Hereda de `Producto` y agrega dos cosas propias. La fecha de vencimiento, obligatoria, que el constructor valida y el detalle muestra. Y una línea de precio oferta, con un 20% de descuento sobre el precio final.

Para discutir entre los dos antes del cierre.

4. Contar las líneas de `Alimento.java` y compararlas con la estimación de la parte 1. ¿Dónde está todo lo que no hubo que escribir?
5. Para calcular la oferta hace falta el precio final. Si se pasa `precio` y el IVA a `protected` para hacer la cuenta directo en `Alimento`. ¿Qué gana? ¿Qué pierde? ¿Alcanzaba con el `getPrecioFinal()` heredado?
6. Nada impide escribir en `Tienda` la línea `new Producto("PRO-001", "Producto generico", 1000)`. Compila y corre. ¿Tiene sentido ese objeto? ¿Qué estaría vendiendo la tienda? Java tiene una herramienta para prohibir exactamente esto, y es el tema de la clase que viene.